SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA15 Cloud Computing and Big Data Analytics

CO2 AT3 - Capstone Cloud Infrastructure Case Analysis and Defense

Editable student template for application-based cloud virtualization and SDDC/cloud operations defense

Assessment Metadata

Course Code	CSA15
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT3 - Capstone Cloud Infrastructure Case Analysis and Defense
Submission Type	Individual digital case analysis and infrastructure defense based on the same capstone title used in CO1, CO2 AT1 and CO2 AT2
Faculty	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Submission Mode	Completed DOCX template and GitHub repository link through Google Classroom

How This Template Works

- Use the same capstone title used in CO1, CO2 AT1 and CO2 AT2.
- Do not recalculate all VM sizing or repeat all configuration screenshots. Summarize the final AT1/AT2 evidence and use it for design defense.
- Answer all five questions with application-oriented reasoning connected to the capstone product.
- Include sustainable cloud operation decisions wherever relevant: right-sized resources, log retention, storage lifecycle, scheduled shutdown, API-call reduction and VM/container/serverless comparison.
- Paste the final topology diagram in the given response area. The sample diagram is only a structure; redraw it for the student's own product.
- Upload this completed document and supporting evidence to GitHub, then submit the repository link through Google Classroom.

Assessment Focus

AT1 answered what VM is required. AT2 showed configuration or simulation evidence. AT3 asks whether the student can defend the complete infrastructure design under application, security, operations and failure conditions.

Student and Capstone Details

Student Name	MUKILAN R
Register Number	192421043
Class / Slot	Slot B

Capstone Title	SkillForge-CO-Attainment Cloud Tutor Using Recommendation Analytics
CO1 Evidence Link	To be added - link to CO1 concept map/presentation repository
CO2 AT1 VM Recommendation	vCPU: 8 RAM: 32 GB Storage: 100 GB SSD VM Class: Two-tier (App VM + managed DB), AI/Analytics workload
CO2 AT2 Evidence Link	To be added - link to CO2 AT2 workbook and evidence repository
GitHub Repository Link	To be added after final GitHub repository is created
Submission Date	Pending - to be recorded at final submission

Evidence Carry-Forward Summary

Evidence Source	Student Entry	How It Supports AT3 Defense
CO1 concept map / presentation	SkillForge concept map and CO1 presentation outlining the recommendation-engine-driven cloud tutor	Establishes the original product scope and cloud service needs referenced throughout AT1-AT3
CO2 AT1 sizing workbook	AT1 sizing workbook: CPU score 9, recommended vCPU 8, RAM 32 GB, storage 100 GB SSD, two-tier design	Provides the quantified sizing baseline defended in Q1 and Q2
VM calibration sheet	Traffic calibration: 1000 registered users, 10% active, peak factor 2, ~10 RPS, 12,500 daily requests	Justifies the workload classification and CPU/RAM scoring used in this defense
CO2 AT2 practical evidence	Local VirtualBox VM (Tiny Core) with snapshot before-update-or-demo; Azure VM and Docker container evidence in progress	Demonstrates practical lifecycle actions (boot, snapshot, planned deployment) referenced in Q2
Capstone architecture decision	Two-tier design: containerized recommendation engine + API VM, managed database and managed storage	Forms the basis for the topology and integration explained in Q3

Q1: Virtualization Value and Capstone Cloud Deployment Summary

Explain your capstone product, expected workload, cloud need and deployment context. Analyze how virtualization improves cost efficiency, resource utilization, workload isolation, infrastructure flexibility and cloud deployment readiness for your product.

Design Point	Student Response
Capstone product and users	SkillForge - an AI-driven Cloud Tutor that tracks CO-attainment and recommends learning content, used by Students, Faculty and Admins
Expected workload	AI/analytics-heavy workload: recommendation inference, CO-attainment dashboards, and moderate REST API traffic (~10 RPS at peak)
Cloud need	Elastic compute for the recommendation engine, a managed database for CO-attainment records, and object storage for course-material uploads
Virtualization value	Virtualization lets one physical host run isolated app, database and AI-service environments without dedicating separate hardware to each, and lets the same VM image be redeployed identically across dev, pilot and production

Cost and utilization benefit	Right-sized burstable VMs avoid paying for idle production-scale capacity during prototyping; the app VM's CPU is shared efficiently across auth, API and dashboard functions instead of running underused dedicated servers
Isolation and flexibility benefit	The recommendation engine runs isolated in its own container so a crash or high load in the AI module cannot take down the core API; VMs/containers can be resized or replaced independently as each module's load changes

Case Analysis Response: Write 8-12 technical lines connecting virtualization value to your capstone product.

Virtualization is what lets SkillForge run several distinct workloads - the API, the database connection layer, and the AI recommendation engine - on infrastructure that is sized for a prototype rather than a full production footprint. Instead of buying dedicated hardware for each module, one virtualized app VM hosts the API while the recommendation engine runs in its own container, and the database sits on a managed service. This keeps cost proportional to actual usage: the AT1 sizing exercise already showed that only a burstable B-series VM is needed for pilot-scale traffic (~10 RPS), not the full 8 vCPU/32GB estimate reserved for later growth. Isolation matters because the recommendation engine's CPU-heavy inference must never be allowed to starve the login/API path that Students and Faculty depend on every day. Flexibility comes from being able to resize, snapshot, or replace any one virtualized component - as demonstrated with the local VM snapshot in AT2 - without touching the others. Together, these properties make SkillForge cloud-deployment ready: the same image and configuration validated locally can move to Azure with only size and network changes, not a redesign.

Q2: Rapid Provisioning, Testing and Virtual Machine Lifecycle Case

Using your CO2 AT1 VM recommendation and CO2 AT2 practical evidence, explain how virtual machines, VM images, templates, snapshots, clones or containers support rapid provisioning, testing, deployment, rollback and maintenance for your capstone product.

Lifecycle Element	AT1/AT2 Evidence Used	Capstone Use / Justification
VM image or base environment	Tiny Core Linux ISO (local VM) and planned Ubuntu Server 24.04 LTS Azure image	Base images give a fast, repeatable starting point so the same OS configuration used in AT2 can be redeployed for the pilot app VM
Template or repeatable setup	Fixed VM naming convention (csa15-vm-<regno>) and resource-group pattern established in AT2	A consistent naming/config template lets new environments (dev, pilot, prod) be created quickly without re-deciding size or network rules each time

Snapshot / rollback	before-update-or-demo snapshot taken on the local VirtualBox VM in AT2	Snapshots before updates/demos let the app VM roll back instantly if a deployment or config change breaks the CO-attainment tracker
Clone / testing copy	Clone not yet attempted in AT2; planned for testing recommendation-algorithm changes	A cloned VM lets the recommendation engine be tested with new models without risking the live pilot environment
Container or VM decision	AT1 Activity 10 VM/container table: recommendation engine and analytics containerized, API on VM, DB/storage managed	Containers give the recommendation engine fast restart and independent scaling; the API stays on a VM for persistent session/auth handling
Maintenance and update plan	AT2 security/cost-control closeout: stop/deallocate the VM after evidence capture	Scheduled snapshot-before-patch and VM shutdown outside test windows keeps maintenance safe and cost-controlled

Case Analysis Response: Explain how lifecycle practices reduce deployment delay and risk.

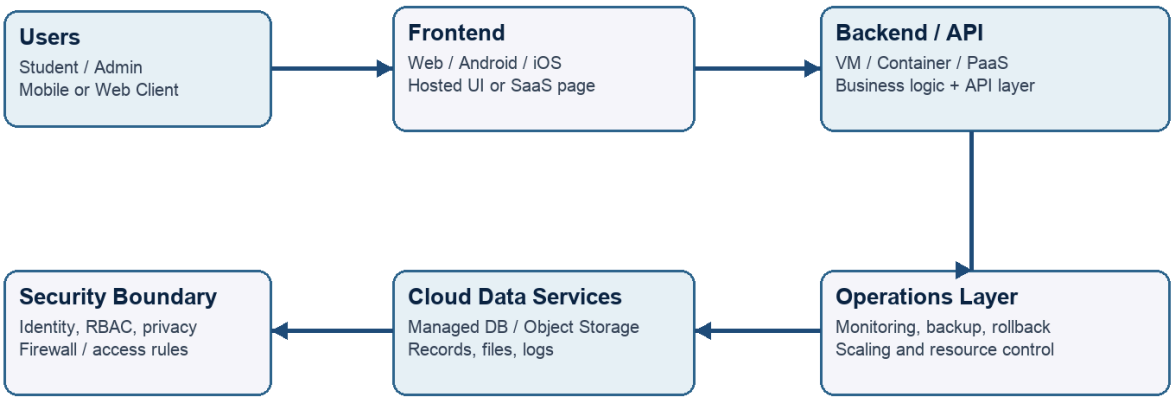
Using a known base image (Tiny Core locally, Ubuntu on Azure) means every new environment starts from an identical, already-verified state, so testing does not have to rediscover basic OS/network issues each time. Snapshots taken before updates or demos (before-update-or-demo) give an instant rollback point, so a bad configuration change costs seconds, not a full rebuild. Clones let the recommendation algorithm be tested against real-feeling data without touching the pilot environment. Because the recommendation engine is containerized rather than baked into the VM, it can be redeployed or rolled back independently of the API VM, which shortens the blast radius and the deployment window for both risk and delay.

Q3: Final Cloud Deployment Topology and SOA/API Service Integration Case

Draw and explain the final deployment topology for your capstone product. Show frontend, backend/API, database, storage, authentication, notification, analytics and any VM/container/managed cloud service components. Explain how SOA/API-based integration connects these modules and supports modular cloud service design.

Capstone Cloud Deployment Topology - Sample Structure

Students should redraw this with their own modules, services, VM/container choices and security boundaries.



AT3 Evidence: final topology + alternatives + failure response + SDDC/cloud operations + green cloud defense.

Architecture Component	Technology / Service	VM / Container / Managed Service / SaaS	Integration Method
Frontend	React/Next.js static web app	Managed Service (static hosting + CDN)	REST API calls over HTTPS to the backend
Backend/API	Node.js/Express REST API	VM (Azure B-series)	Exposes REST endpoints consumed by the frontend and internally calls the recommendation container
Database	Managed relational database (e.g. Azure Database for PostgreSQL)	Managed Service	Backend connects via a private connection string; the recommendation container reads via API, not direct DB access
Storage	Azure Blob Storage / object storage	Managed Service	Backend generates signed URLs for upload/download of course PDFs
Authentication	JWT-based auth service	Managed Service	Frontend and backend validate JWT tokens on every API call
Notification	Lightweight notification/queue worker	Container	Triggered by backend events (quiz submitted, recommendation ready) via a message queue
Analytics / AI module	Python recommendation engine (embedding/ML model)	Container	Backend calls the recommendation container's internal REST endpoint; results cached and served to the dashboard

Paste or draw the final capstone deployment topology diagram here.

SOA/API Integration Explanation: Explain how services communicate and why modular service design is useful.

Each module exposes a narrow REST API rather than sharing a database or file system directly: the frontend only talks to the backend API, the backend only talks to the recommendation container's API and the managed database, and notifications are triggered by backend events rather than polling. This SOA/API-based integration means any one service (for example the recommendation engine) can be redeployed, rescaled or replaced without the frontend or other services needing to change, as long as the API contract stays the same. It also lets each component be secured, scaled and monitored on its own terms - the AI container can scale for inference load while the API VM stays fixed-size for steady session traffic.

Q4: Design Alternatives, Identity, Access and Federated Cloud Privacy Case

Compare at least two deployment alternatives for your capstone product, such as single VM, multi-tier VM, containerized deployment, managed backend, federated cloud or hybrid cloud model. Justify the selected design by analyzing identity management, role-based access control, federated access, data sharing, privacy protection and security risks.

Deployment Alternative	Advantages	Limitations / Risks	Identity and Privacy Impact
Single VM monolith (all modules on one VM)	Simple to deploy and cheap for a prototype; minimal network complexity	Recommendation engine competes with the API for CPU under load; single point of failure; harder to patch independently	All auth/session logic and CO-attainment data live on one host, increasing blast radius if that VM is compromised
Two-tier containerized design (API VM + containerized recommendation engine + managed DB/storage)	Independent scaling of the AI module; managed DB improves backup/reliability; strong workload isolation	Slightly higher setup complexity; needs container orchestration and inter-service authentication	Managed DB and storage apply provider-level encryption/access controls, and service-to-service calls use their own scoped credentials rather than shared VM-level secrets
Two-tier containerized design (selected)	Matches the AT1 sizing/VM-container decisions already validated in AT2	Selected because it isolates the recommendation engine and puts the highest-risk data (student records) on a managed, hardened service	Role-based access (Student/Faculty/Admin) enforced at the API layer via JWT claims; managed DB access restricted to the backend service identity only

Security Area	Student Decision
User roles	Student, Faculty, Admin (carried from CO2 AT1 Activity 1)
Authentication method	JWT-based session tokens issued after login for the API; SSH-key authentication used for VM administration
Role-based access control	Students view/take quizzes and recommendations; Faculty upload content and monitor CO-attainment; Admin manages courses, COs and user accounts
Federated access / partner access	Not required at prototype stage; institutional SSO / federated login via the college identity provider is considered for a future production phase
Sensitive data handled	Student personal details, quiz scores, CO-attainment records and uploaded course materials
Privacy protection decision	Student records and quiz scores are stored only in the managed database with encryption at rest; API responses are scoped so students see only their own data,

	and Faculty/Admin views are restricted by role
Case Analysis Response: Defend why the selected design is better for identity, access and privacy. The two-tier containerized design is better for identity, access and privacy because it keeps the highest-risk data - student CO-attainment records - inside a managed database with provider-level encryption and access control, rather than on a general-purpose VM disk that also runs the API and other logic. Role-based access is enforced centrally at the API layer using JWT claims, so Students, Faculty and Admin each see only what their role permits regardless of which backend module handles the request. Because the recommendation engine is a separate service, it can be given its own scoped credentials to read only what it needs, instead of inheriting broad database access the way a single-VM monolith would. This reduces the blast radius of any one compromised credential and keeps privacy-sensitive data under the tightest, most auditable boundary in the design.	

Q5: Software-Defined Datacenter and Cloud Operations Defense Case

Analyze how your infrastructure will respond to operational and failure scenarios such as VM failure, database growth, traffic spike, credential leakage, service outage, resource contention or cost increase. Explain how SDDC concepts and cloud operations practices such as automated provisioning, software-defined networking, software-defined storage, monitoring, logging, backup, rollback, scaling, resource control and maintenance improve the final infrastructure design. Include Green Computing decisions such as resource right-sizing, storage lifecycle control, log retention, API-call reduction, scheduled shutdown, VM/container/serverless comparison and carbon/energy proxy indicators where relevant.

Operational / Failure Scenario	Expected Impact	SDDC / Cloud Operations Response	Evidence or Control
VM failure	API becomes unreachable; recommendation and dashboard features stop	Redeploy from the saved VM image/template; app-tier statelessness lets a new VM instance take over quickly	AT2 snapshot/rollback evidence and the planned Azure VM configuration table
Database growth	Slower CO-attainment queries and rising storage cost	Managed database with automated storage scaling and archiving of old quiz records; add read replicas if query load grows	AT1 storage calibration (100 GB with a 30% backup buffer) as the initial baseline
Traffic spike	Increased latency during peak study hours (per the AT1 peak factor of 2)	Autoscale or scale up the API VM tier during evening peak; the recommendation container scales independently since it is the heavier workload	AT1 traffic calibration (~10 RPS peak, peak factor 2)
Credential leakage	Unauthorized access to student data or cloud resources	Rotate SSH keys/JWT secrets immediately, revoke the exposed key/token, review access logs for anomalies	AT2 checkpoint confirming no passwords/keys were uploaded
Service outage	Students/Faculty cannot log in or submit assessments	Health checks with automatic restart; maintain a recent snapshot/image to redeploy quickly	AT2 VM snapshot practice (before-update-or-demo)
Resource contention	Recommendation inference slows down API responses if sharing one VM	Recommendation engine isolated as its own container/service so CPU-heavy inference cannot starve the core API	AT1 VM/container decision table separating the AI module from the API
Cost increase	Rising Azure credit consumption from VMs left running	Scheduled shutdown of non-production VMs outside lab/demo hours, right-sized burstable VM tier, monitoring the Azure cost dashboard	AT2 Azure Education overview showing credit balance, and the cost-control closeout checklist

Operational Readiness Area	Student Plan
Automated provisioning	Infrastructure steps (resource group, VM, image, ports) follow a fixed procedure from CO2 AT2 that can be scripted with Azure CLI/ARM templates for repeatable provisioning
Software-defined networking	Network security group rules restrict inbound traffic to SSH (22) only by default, with HTTP opened only for temporary test windows, as practiced in CO2 AT2
Software-defined storage	Managed object storage and database storage scale independently of the VM disk, avoiding manual disk resizing
Monitoring and logging	Azure VM diagnostics/metrics and application logs (API request logs, container logs such as docker logs) are monitored for errors and load
Backup and rollback	VM snapshots taken before updates/demos (as in CO2 AT2) plus

	automated backups on the managed database
Scaling and elasticity	The API VM can scale up (larger burstable size) or out (multiple instances behind a load balancer); the recommendation container scales independently based on inference load
Resource control and maintenance	Non-production VMs are stopped/deallocated after evidence capture; scheduled patching windows are preceded by a snapshot
Green Computing and sustainability control	A right-sized B-series VM is used instead of over-provisioned production sizing for the prototype; idle resources are shut down and retention limits reduce stored data volume

Green Cloud Decision	Student Justification
VM right-sizing from CO2 AT1	CO2 AT1 recommended 8 vCPU/32GB for the full production AI workload; CO2 AT2 intentionally uses a much smaller B-series VM for lab/prototype evidence, avoiding wasted capacity while the product is still small-scale
VM vs container vs serverless choice	The recommendation engine and background jobs run in containers (fast start, scale only when needed) rather than always-on VMs; a serverless function is considered for infrequent notification triggers to avoid idle compute
Scheduled shutdown / idle-resource control	The Azure VM is stopped/deallocated after each evidence session (per the CO2 AT2 closeout checklist); the local VirtualBox VM is run only during active lab work
Log and file retention policy	API and container logs are retained for a limited window (e.g. 30 days) rather than indefinitely; old quiz/session logs are archived or deleted after the retention period
Storage lifecycle and backup cleanup	Uploaded course files move to cooler/archive storage tiers after a set age; unused snapshots and clones are deleted once their rollback/testing purpose is served
API-call or polling reduction	Dashboard and recommendation refresh use event-driven triggers/caching instead of constant

	polling, reducing unnecessary compute cycles
Carbon/energy proxy indicator to track	VM/container CPU-hours per month and Azure cost-per-month, tracked via the Azure cost dashboard shown in CO2 AT2, are used as a proxy for energy use
Final Infrastructure Defense Statement: State whether the design is ready for prototype, pilot or production. Defend reliability, security, SDDC/cloud operations and Green Computing improvements required next. The current design is ready for prototype and early pilot use, but not yet for production. Reliability is supported by VM snapshots, planned managed-database backups and stateless API design, but true production would need automated health checks, multi-instance redundancy and tested failover, none of which exist yet. Security is reasonable for a lab/pilot (SSH-only access, JWT auth, role-based access, no secrets in evidence), but production would need federated institutional login, secret rotation automation and a formal incident-response process. SDDC/cloud-operations maturity is currently manual - snapshots, shutdowns and scaling decisions are performed by hand rather than scripted - so the next step is to codify provisioning (IaC), monitoring/alerting and autoscaling rules. Green Computing practice already includes right-sizing and idle-shutdown habits from AT2, but log retention, storage lifecycle and API-call reduction remain manual policies that should be automated before scaling to more users. In summary: solid prototype-stage defense, with a clear, prioritized list of reliability, security,	

operations and sustainability improvements required before pilot or production rollout.

CO2 Reflection and Individual Defense

This section must be individually written. It should show what the student understands from the complete CO2 chain: infrastructure sizing, practical configuration evidence and final operations defense.

Reflection Prompt	Student Response
What is the strongest infrastructure decision in your capstone design?	Isolating the recommendation engine as its own container, separate from the core API VM, since it is both the differentiating feature and the heaviest, most variable workload
Which CO2 concept became clearer after AT1, AT2 and AT3?	How sizing math (AT1), practical configuration/security evidence (AT2) and full lifecycle/operations defense (AT3) connect into one continuous infrastructure story rather than three separate exercises
Which risk is most serious for your capstone: failure, cost, privacy, scaling or sustainability?	Cost and resource contention are the most immediate risks at prototype stage, since AI inference is CPU-heavy and easy to over- or under-provision; privacy becomes more serious once real student data is loaded
What evidence would you show during viva to defend your infrastructure design?	The CO2 AT1 sizing workbook (CPU/RAM/storage calculations), CO2 AT2 VM/snapshot/Docker screenshots, and this AT3 topology and operations tables
What will you improve before final capstone deployment?	Completing the pending Azure VM deployment and Docker evidence, adding real monitoring/alerting, and turning the scheduled-shutdown and log-retention policies into actual automation rather than manual steps

Student-Facing Assessment Criteria

Criteria	Descriptor
Capstone continuity	The answer uses the same capstone title and connects CO1, CO2 AT1 and CO2 AT2 evidence.
Virtualization and lifecycle reasoning	VM, image, snapshot, clone, container or managed-service decisions are technically justified.
Topology and integration clarity	Architecture diagram and SOA/API explanation show clear module interaction.
Identity, access and privacy	Roles, authentication, RBAC, federated access, sensitive data and privacy decisions are defended.
SDDC/cloud operations	Failure response, monitoring, logging, scaling, backup, rollback and resource control are addressed.
Green Computing defense	Right-sizing, retention, API efficiency, idle-resource control and sustainable cloud choices are explained.
Submission quality	GitHub, template completion, evidence references and individual reflection are complete.

GitHub and Google Classroom Submission

Repository Item	Expected Content
README.md	Capstone title, student details, final VM recommendation, deployment topology summary and AT3 evidence index.
Completed AT3 document	This completed editable DOCX or exported PDF if instructed.
Diagram image	Final topology diagram if created separately.
Evidence references	Links or screenshots from CO1, CO2 AT1 and CO2 AT2 as needed.
Green Computing note	Short README section explaining sustainable cloud choices and resource-control decisions.
Google Classroom	Submit the GitHub repository link through GCR.

Student Assurance

I confirm that this case analysis, design reasoning, topology diagram and final defense statement were prepared individually by me for the stated capstone title. I have not uploaded passwords, tokens, private keys or sensitive account information.

Student Name	MUKILAN R
Register Number	192421043
Signature	
Date	

Faculty Verification

Faculty Name	Dr C. Rajesh Babu
Employee ID	SSETSCS415
Constructive Feedback / Remarks	
Strength Observed	
Improvement Suggested	
Signature / Date	